Indice

DDTA - Working Metamodel - Macro Requirement

> **NON CANONICAL WORKING DRAFT**
>
> This file defines the general document metamodel. ThreatForge-specific observations are excluded and belong to the case-study material.

1. Scope

The metamodel describes the knowledge of a governed project before deciding its concrete Doc-as-Code serialization and before deciding how a software engine validates or assists authoring.

The governed product remains documentation-as-code: versionable, diffable, reviewable files (for example YAML registries and Markdown bodies). A software tool may apply and verify the metamodel, but it is not the subject described by the project's documents.

Layer separation

1. **Domain metamodel** - meaning of documents, entities and relations.
2. **Doc-as-Code representation** - how that meaning is serialized in files.
3. **Engine** - how tools load, validate, correlate, project and assist those files.

2. Principles already fixed

Principle 1 - Progressive specialization and semantic containment

Governed project knowledge is represented as a hierarchy of progressively specialized governed documents. A Macro Requirement establishes a macro-level subject and purpose; Decisions narrow that subject through explicit choices; Functional Requirements translate those choices into independently verifiable obligations; Specialized Requirements add domain-specific constraints to one Functional Requirement.

Hierarchical parentage represents semantic containment and remains distinct from non-hierarchical traceability, evidence and provenance relations.

Principle 2 - Identity independent from textual encoding

Every governed entity has a stable identity independent from the textual encoding of that identity. Hierarchy is established by explicit semantic relationships. Composite identifier formats may mirror ancestry but must not be the authoritative source of topology.

3. Progressive disclosure by competence

The hierarchy increases both specificity and the amount of vertical competence required from the reader.

- **Macro Requirement** - understandable by stakeholders who understand the project domain, including non-software stakeholders.
- **Decision** - requires more knowledge of the solution space and project choices.
- **Functional Requirement** - precise, testable project behaviour or outcome.

- **Specialized Requirement** - vertical competence such as governance, security, quality or another project-specific discipline.
- **Analysis models and methodology-specific records** - specialist analytical concepts introduced only when needed.

Executive-map criterion

Reading only the **title and Intent** of all Macro Requirements of a governed project must produce a recognizable summary of:

- what the project is trying to realize;
- what general value it produces;
- why its main blocks of work exist.

This criterion applies to the **governed project**, not to the tool used to govern it.

4. Domain vocabulary versus metamodel vocabulary

The metamodel must not ban technical words merely because they are technical.

A term belongs in an MR when it is part of the **application domain** and is reasonably understandable to the stakeholders of that domain. For example, a cryptography project may naturally use security terminology; a medical imaging project may naturally use imaging terminology; a facial-recognition access project may naturally use terms such as facial recognition, biometric data or ML model.

What the MR must not be forced to contain are technical concepts introduced **only because the documentation model or an analysis method expects them**, such as:

- component inventories;
- data-flow decompositions;
- trust boundaries;
- protocol details;
- attack-surface classifications;
- STRIDE/LINDDUN categories;
- method-specific threat or privacy taxonomies.

Those concepts normally emerge later, when Decisions, Requirements, Base Analysis or methodology-specific analysis require them.

5. Macro Requirement definition

A Macro Requirement identifies one major project concern or macro-capability. It explains why that concern matters, who is affected, and the general result the project seeks at that level.

It is the root document type of the governed-document hierarchy.

Fundamental question

> **Which major project concern or result are we governing, why does it matter, and who is involved?**

6. Candidate information model

Concept	State	Purpose
`id`	required	stable identity, history and traceability
`title`	required	concise name understandable within the project domain
`lifecycle`	required	current/historical state of the document
`intent`	required	concise explanation of why the MR exists and why it matters

Concept	State	Purpose
`context`	required	minimum background needed to understand the concern
`stakeholders`	required	people/groups that benefit, use, govern or are affected
`objective`	candidate required	macro result sought; not a list of atomic obligations
`scope`	open semantic decision	boundary of the macro concern; exact form still under study
`assumptions`	optional	facts accepted as true that materially affect the whole MR
`constraints`	optional	limits that materially constrain the whole MR
`non_goals`	open semantic decision	plausible expectations deliberately outside the MR, if useful

Concision

A Macro Requirement must be effective rather than verbose. It should not become a checklist of implementation rules.

- `title`: short noun phrase or short descriptive title;
- `intent`: normally 1-2 sentences;
- `context`: only the background needed to interpret the MR;
- `stakeholders`: macro stakeholder roles, not a technical inventory;
- `objective`: normally a small number of high-level results;
- assumptions/constraints: only if they affect the whole branch below the MR.

7. Decisions already closed

7.1 Objective is not a list of `must`

The MR does not own atomic, independently verifiable obligations. Those belong to Requirements. `Objective` describes a macro result, not a long normative checklist.

7.2 Stakeholders are semantically required

The MR must identify the stakeholder groups relevant to its macro concern. Their names may be technical when that is normal domain language, but they should be meaningful to the project community rather than internal implementation labels.

7.3 Assumptions and Constraints are distinct

- **Assumption**: something currently treated as true for the branch of work.
- **Constraint**: a limit within which the branch must operate.

Both are optional and belong in the MR only when they materially influence the whole branch below it.

7.4 Parent and children

`MacroRequirement.parent = null`.

A Macro Requirement may own `0..*` Decisions. The child collection is derived from `Decision.parent -> MacroRequirement`; it is not a second topology authority.

7.5 Cross-MR dependency

A Macro Requirement may have a non-hierarchical `dependsOn -> MacroRequirement [0..*]` relation when another macro concern is a genuine dependency. The exact Doc-as-Code serialization is deferred.

7.6 No Acceptance Criteria at MR level

The MR does not contain atomic acceptance criteria. Evidence of satisfaction emerges from its descendant Requirements and their verification evidence.

7.7 Progressive technicality

The MR should be understandable by stakeholders competent in the **project domain** without requiring them to understand implementation decomposition or analysis methodology. More specialized technical knowledge is introduced as documentation descends.

7.8 Technical and analytical decomposition normally emerges later

The MR must not be structurally required to enumerate components, services, APIs, data flows, trust boundaries, attack surfaces or threat categories. Those normally emerge at lower documentation or analysis levels.

This does **not** prohibit domain-specific terminology in an MR.

8. MR invariants

1. **Root hierarchy** - MR has no parent.
2. **Stable identity** - identity is distinct from its textual encoding.
3. **Single macro concern** - the MR expresses one coherent macro responsibility.
4. **Child containment** - descendants progressively narrow the MR rather than introduce unrelated scope.
5. **No implementation choice** - choices of solution belong to Decision unless they are intrinsic to the project domain itself.
6. **No requirement-level obligation** - atomic testable obligations belong to Requirements.
7. **Broad audience within the domain** - title and Intent can be understood by non-software stakeholders who understand the project domain.
8. **Project-map usefulness** - titles + Intent of all MR summarize the project and its major work blocks.
9. **Explicit dependencies** - cross-MR dependencies are not hidden as pseudo-hierarchy.
10. **Analysis enabling, method agnostic** - MR provides project knowledge but does not prescribe a threat-analysis paradigm.
11. **Architecture resilience** - changing a lower-level architecture choice should not normally force the MR to be rewritten if the macro project concern has not changed.

9. Semantic decisions still open

These points must be tested with concrete examples before the MR model is frozen:

9.1 Intent versus Objective

We need examples to determine whether both concepts add distinct information or whether they create unnecessary duplication.

Candidate distinction:

- `Intent` - why this MR exists / why it matters;
- `Objective` - what macro result we want to obtain.

This distinction is **not yet frozen**.

9.2 Meaning and shape of Scope

We need examples to determine what `scope` should actually express and whether `included`, `excluded` and `non_goals` are all necessary or partly redundant.

The goal is not to create three places that repeat the same boundary in different words.

9.3 When to split one MR into multiple MRs

A candidate rule is that an MR should be split when it contains multiple independent macro objectives, substantially different stakeholder concerns, or Decisions that do not share a coherent Intent. This must be tested against examples before becoming an invariant.

10. Doc-as-Code mapping deliberately deferred

The following remain representation questions, not domain-model decisions:

- where each MR concept lives between registry and Markdown body;
- whether `parent = null` is serialized or implicit;
- concrete representation of stakeholders, assumptions and constraints;
- prose versus list representation of Objective;
- concrete representation of `dependsOn`;
- final Markdown heading names;
- deterministic checks that can enforce the semantic contract without pretending to validate human meaning.

STOP: Decision is not modeled in this document yet.